High-Level Optimization of Abstract Data Types

Anthony Hunt and Emil Sekerinski

ONPLS 2026
McMaster University
June 1, 2026

Programming with Abstract Data Types

Programming With Abstract Data Types

- System specifications use sets and relations
  - Expressive and provable
- Semantics from mathematics

Syntax	Label	Syntax	Label
$set(T)$	Unordered, unique collection	$S \cup T$	Union
$S \leftrightarrow T$	Relation, $set(S \times T)$	$S \cap T$	Intersection
$S \rightarrow \mathbb{N}$	Bag/Multiset	$S \setminus T$	Difference
$\mathbb{N} \rightarrow S$	Sequence	$S \times T$	Cartesian Product
$\{x, y, \ldots\}$	Set Enumeration	$dom(R)$	Domain
$\{x \cdot P \mid E\}$	Set Comprehension	$R[S]$	Image

Jean-Raymond Abrial. *Modeling in Event-B: system and software engineering*. Cambridge University Press, 2010.
David Gries and Fred Schneider. *A Logical Approach to Discrete Math*. Springer New York, 1993.

Example - Visitor Information System

- Academics attend workshops throughout a conference
- Every workshop must be held in one room
- Visitors may attend at most one workshop at a time

$$Visitor = \{Adrian, Kevin, Charlie, Michael\}$$
$$Room = \{1305, 1306, 1301\}$$
$$Workshop = \{ESOP, FASE, FoSSaCS\}$$

Total Injective Function

$$location : Workshop \rightarrowtail Room$$
$$attends : Visitor \nrightarrow Workshop$$

Partial Function

Then, the number of meals to prepare for a specific $room$ would be:

$$card((location^{-1} \circ attends^{-1})[\{room\}])$$

Relational Image

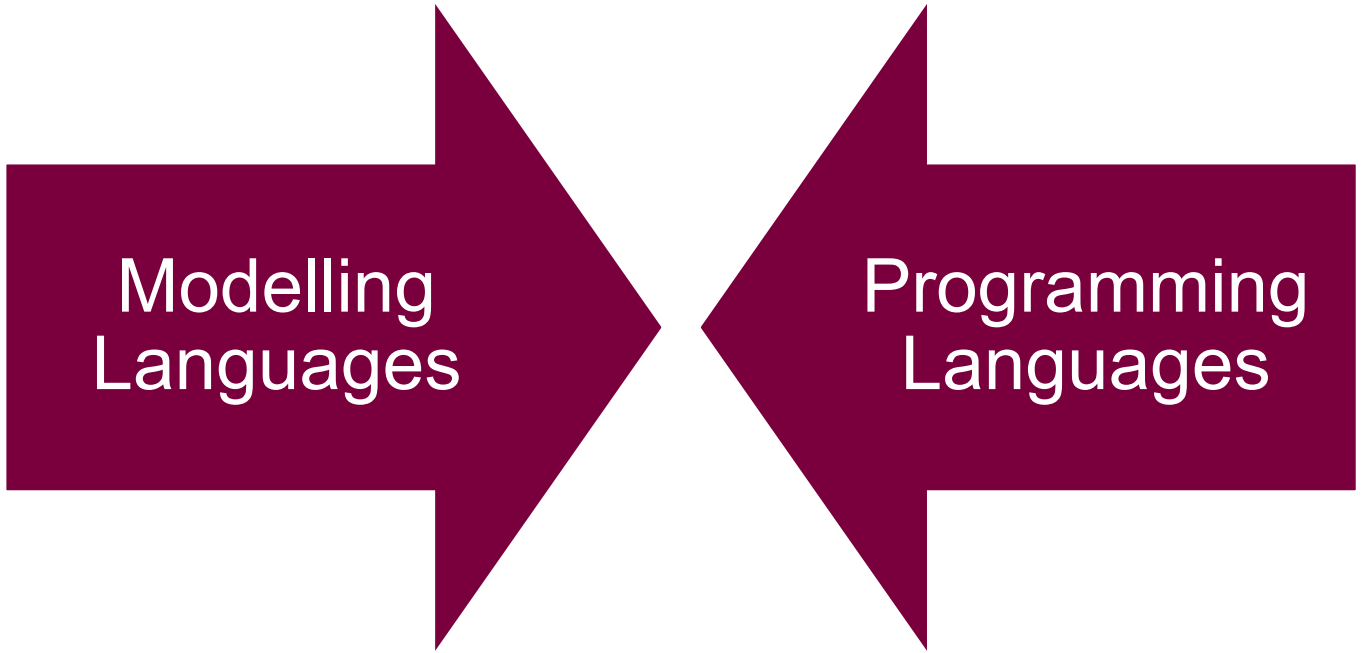
Abstract Data Types

In Programming and Modelling Languages

		Sets				Relations									Efficiently Executable	Implementation Free
		$\cup$	$\cap$	$\times$	$\{x \cdot P \mid E\}$	dom	$\lhd$	$\cup$	$\cap$	$\lhd$	$\circ$	$R[S]$	R^{-1}	Multiplicity		
Programming Languages	**Python**	✓	✓	✓	✓	✓	✗	✗	*	✓	✗	*	✗	✗	*	✗
	Haskell	✓	✓	✓	*	✓	*	✗	✓	✓	*	*	✓	*	✓	✗
	Rust	✓	✓	*	*	✓	✗	✗	✗	*	✗	✓	✗	✗	✓	✗
	C	*	*	*	✗	*	*	*	*	*	*	*	*	✗	✓	✗
Modelling Languages	**SetL**	✓	✓	*	✓	✓	✓	✓	✓	*	*	✓	*	*	*	*
	UML	✓	✓	✓	✗	✓	✗	✗	✗	✓	✗	✓	✗	✓	*	✓
	Event-B	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	*	✓
	Alloy	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✗	✓

Goals

- Syntax close to discrete mathematics
- Simple, small, and usable
- High-level set-theory optimization
- Efficient in memory and time



Related Work

- ProB animation engine for Event-B [1]
- GHC rewrite rules [2]
- Rewrite systems for set-theory focused optimization [3,4,5]
- SETL data type *lowering* from abstract types to suitable concrete structures [6,7]
- Cozy: data structure synthesis from specifications [8]

[1] Michael Leuschel. *Programming in B: Sets and Logic all the Way Down.* In: LPOP 2022.
[2] Simon Peyton Jones, Andrew Tolmach, and Tony Hoare. *Playing by the rules: rewriting as a practical optimisation technique in GHC*. In: 2001 Haskell Workshop. ACM SIGPLAN. Sept. 2001.
[3] Maximiliano Cristia and Gianfranco Rossi. *{log}: Programming and Automated Proof in Set Theory.* In: LPOP 2022.
[4] Elco Visser, Zine-el-Abidine Benaissa, and Andrew Tolmach. Building program optimizers with rewriting strategies. In: ICFP 1998.
[5] Douglas R. Smith and Stephen J. Westfold. *Transformations for Generating Type Refinements*. In: FM 2019.
[6] Edmond Schonberg, Jacob T. Schwartz, and Micha Sharir. 1979. *Automatic data structure selection in SETL*. In: POPL 1979.
[7] Stefan M. Freudenberger, Jacob T. Schwartz, and Micha Sharir. *Experience with the SETL Optimizer.* Association for Computing Machinery, 1983
[8] Calvin Loncaric, Michael D. Ernst, and Emina Torlak. *Generalized Data Structure Synthesis*. In: ICSE 2018.

Optimizing Abstract Expressions

Example - Optimizing Abstract Data Type Operations (in Python)

$(S \cup T) \cap V$

```
# Python translation:
ret = set()
for x in S:
    ret.add(x)
# Step 1: len(ret) == len(S)
for x in T:
    ret.add(x)
# Step 2: len(ret) <= len(S) + len(T)
for x in ret:
    if x not in V:
        ret.remove(x)
# Step 3: len(ret) <= len(V)
```

What if $len(S) + len(T) > len(V)$?

$(S \cap V) \cup ((T \setminus S) \cap V)$

```
# New Python translation:
ret = set()
for x in S:
    if x in V:
        ret.add(x)
# Step 1: len(ret) <= min(len(S), len(V))
for x in T:
    if x not in S and x in V:
        ret.add(x)
# Step 2: len(ret) <= min(len(S) + len(T), len(V))
```

ret never grows larger than needed

Optimizing Expressions - What do we need to know?

- Simple syntax must have rigid semantics and well-definedness conditions
  - Programmer should focus on the correctness of the abstract, provable program
- Abstract data types can be represented by a variety of concrete implementations
  - Unique optimization paths and atomic operations
  - Structural limitations
  - Operation advantages

Atomic Operations	
Bitsets	**Hashsets**
$S \cup T$	$x \in S$
$S \cap T$	$S.add(x)$
S^C	$S.remove(x)$

Structural Limitations

Hashmaps: many-to-one relationships
Bitsets: sets of integers within a small range

Preferred operations

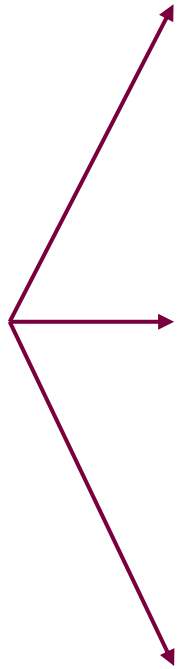
Bitsets: intersections, unions
Arrays: direct lookup, appending

Optimizer Design Considerations

- Directed by formal set theory semantics
- Refinement-based type system directs application of optimizations
- Use a term rewriting system:
  - Simplify operations
  - Select concrete data representations from refined types
  - Generate efficient code for every concrete representation and operation sequence

(Some) Concrete Data Types of the Visitor Information System

$$location : Workshop \rightarrowtail Room$$
$$attends : Visitor \nrightarrow Workshop$$



$$location : set[tuple[Workshop, Room]]$$
$$attends : set[tuple[Visitor, Workshop]]$$

$$location : BiMap[Workshop, Room]$$
$$attends : BiMap[Visitor, Workshop]$$

$$location : HashMap[Workshop, Room]$$
$$attends : HashMap[Visitor, Workshop]$$

Naïve Implementation of the Visitor Information System

Concrete Types

$location: set[tuple[Workshop, Room]]$
$attends: set[tuple[Visitor, Workshop]]$

Imperative Interpretation

$card((location^{-1} \circ attends^{-1})[\{room\}])$

$l := inverse(location)$
$a := inverse(attends)$
$c := compose(l, a)$

…

Library

proc $inverse(r)$:
  $r' := \{\}$
  for $k, v \in r$ **do**
    $r' := r' \cup \{(v, k)\}$
  return r'

Runtime: $O(n)$
Memory: up to $O(n)$

proc $compose(r_1, r_2)$:
  $r' := \{\}$
  for $k, v \in r_1$ **do**
    for $v', t \in r_2$ **do**
      if $v = v'$ **then**
        $r' := r' \cup \{(k, t)\}$
  return r'

Runtime: $O(n^2)$
Memory: $O(n)$

Hashmap-based Expression Optimizer – Comprehension Construction

Set-typed variables and literals are decomposed into set comprehensions

Rewrite Rules	
Predicate Operations	$S \cup T \rightsquigarrow \{x \cdot x \in S \vee x \in T \mid x\}$
	$S \cap T \rightsquigarrow \{x \cdot x \in S \wedge x \in T \mid x\}$
	$S \setminus T \rightsquigarrow \{x \cdot x \in S \wedge x \notin T \mid x\}$
Membership Collapse	$x \in \{E \mid P\} \rightsquigarrow \exists y \cdot P \wedge x = E$
Image	$R[S] \rightsquigarrow \{y \cdot \exists x \cdot x \mapsto y \in R \wedge x \in S \mid y\}$
Composition	$x \mapsto z \in R \circ Q \rightsquigarrow x, z \cdot \exists y, y' \cdot x \mapsto y \in R \wedge y' \mapsto z \in Q \wedge y = y'$
Inverse	$x \mapsto y \in R^{-1} \rightsquigarrow y \mapsto x \in R$
Cardinality	$card(S) \rightsquigarrow \sum x \cdot x \in S \mid 1$

Post-condition:

- All set-like terms must be comprehensions

$Workshop \rightarrowtail Room$
One-to-One Total Function

$Visitor \nrightarrow Workshop$
Many-to-one Partial Function

$$numMeals = card((location^{-1} \circ attends^{-1})[\{room\}])$$

Image

$$numMeals = card(\{\boldsymbol{v} \cdot \exists \boldsymbol{r} \cdot \boldsymbol{r} \mapsto \boldsymbol{v} \in (location^{-1} \circ attends^{-1}) \wedge \boldsymbol{r} \in \{\boldsymbol{room}\} \mid \boldsymbol{v}\})$$

Composition, Inverse

$$numMeals = card(\{v, \boldsymbol{r} \cdot \exists \boldsymbol{p}, \boldsymbol{p}' \cdot \boldsymbol{v} \mapsto \boldsymbol{p}' \in \boldsymbol{attends} \wedge \boldsymbol{p} \mapsto \boldsymbol{r} \in \boldsymbol{location} \wedge \boldsymbol{p} = \boldsymbol{p}' \wedge r \in \{room\} \mid v\})$$

Cardinality, Membership

$$numMeals = \sum v, r \cdot \exists p, p' \cdot v \mapsto p' \in attends \wedge p \mapsto r \in location \wedge p = p' \wedge r = room \mid \mathbf{1}$$

Hashmap-based Expression Optimizer – Generator Selection

Selecting element generators for use as iterables in generated for-loops

Rewrite Rules	
Generator Selection	$\bigwedge P_i \rightsquigarrow P_g \wedge \bigwedge_{P_i \neq P_g} P\_i$ • P_g is of the form $x \in S$ • x is the quantifier's bound variable • S is a set-like term

- In the case of multiple candidate generators, selection is based on heuristics

$$numMeals = \sum v, r \cdot \exists p, p' \cdot v \mapsto p' \in attends \wedge p \mapsto r \in location \wedge p = p' \wedge r = room \mid 1$$

Generator Selection

$$numMeals = \sum \boldsymbol{v, r} \cdot \exists p, p' \cdot \boldsymbol{v \mapsto p' \in attends} \wedge \boldsymbol{p \mapsto r \in location} \wedge p = p' \wedge r = room \mid 1$$

Post-condition:

- All set-like terms must be comprehensions
- Quantifier predicates are in DNF
  - Guaranteed top-level-∨ operation
- One bound variable per generator
- All predicate ∨-clauses have an assigned generator

Hashmap-based Expression Optimizer – Loop Lowering

Start lowering expressions into imperative-like loops

Rewrite Rules		
Quantifier Generation	$\oplus E \mid P$	$\rightsquigarrow$ $a := identity(\oplus)$ **loop** P **do** $a := a \oplus E$
Chained Generators	**loop** $G_1 \wedge G_2$ **do** $a := a \oplus E$	$\rightsquigarrow$ **loop** G_1 **do** **loop** G_2 **do** $a := a \oplus E$
Disjunct Generators	**loop** $G_1 \vee G_2$ **do** $a := a \oplus E$	$\rightsquigarrow$ **loop** G_1 **do** $a := a \oplus E$ **loop** $G_2 \wedge \neg G_1$ **do** $a := a \oplus E$

Post-condition:

- No quantifiers exist within the AST
- No ∨-operators exist within a ***loop***'s predicate
- All predicate ∨-clauses have an assigned generator

- $\oplus$ is any of $\{\ldots\}$, Σ, Π, $\cup$, $\cap$

$$numMeals = \sum \boldsymbol{v}, \boldsymbol{r} \cdot \exists p', p \cdot \boldsymbol{v \mapsto p' \in attends} \wedge \boldsymbol{p \mapsto r \in location} \wedge r = room \wedge p = p' \mid 1)$$

Quantifier Generation

$$numMeals := 0$$
$$\textbf{loop } \boldsymbol{v \mapsto p' \in attends} \wedge \boldsymbol{p \mapsto r \in location} \wedge p = p' \wedge r = room \textbf{ do}$$
$$numMeals := numMeals + 1$$

Chained Generators

$$numMeals := 0$$
$$\textbf{loop } \boldsymbol{v \mapsto p' \in attends} \textbf{ do}$$
$$\textbf{loop } \boldsymbol{p \mapsto r \in location} \wedge r = room \wedge p = p' \textbf{ do}$$
$$numMeals := numMeals + 1$$

Hashmap-based Expression Optimizer – Relation Subtyping Simplification

Eliminate unnecessary loops

Rewrite Rules		
Restricted Generator	**loop** $x \mapsto y \in R \wedge x = x' \wedge y = y'$ **do** body • R is total • R is one-to-one	$\rightsquigarrow$ **if** $R(x') = y'$ **then** body

One-to-one Total Function

$numMeals := 0$
loop $v \mapsto p' \in attends$ **do**
  loop $p \mapsto r \in location \wedge r = room \wedge p = p'$ **do**
    $numMeals := numMeals + 1$

$\downarrow$

$numMeals := 0$
loop $v \mapsto p' \in attends$ **do**
  if $location(p') = room$ **then**
    $numMeals := numMeals + 1$

Post-condition:

- No quantifiers exist within the AST
- No ∨-operators exist within a ***loop***'s predicate
- All predicate ∨-clauses have an assigned generator

Hashmap-based Expression Optimizer – Loop Code Generation

Lower into imperative instructions

Rewrite Rules	
Conjunct Conditional	$\textbf{loop}\ P_g \wedge \bigwedge P_i\ \textbf{do}\ body \rightsquigarrow \textbf{if} \bigwedge_{free(P_i)} P_i\ \textbf{then}\ \textbf{for}\ P_g\ \textbf{do}\ \textbf{if} \bigwedge_{\neg free(P_i)} P_i\ \textbf{then}\ body$

$$
\begin{array}{l}
numMeals \coloneqq 0 \\
\textbf{loop}\ v \mapsto p' \in attends\ \textbf{do} \\
\quad \textbf{if}\ location(p') = room\ \textbf{then} \\
\quad\quad numMeals \coloneqq numMeals + 1
\end{array}
$$

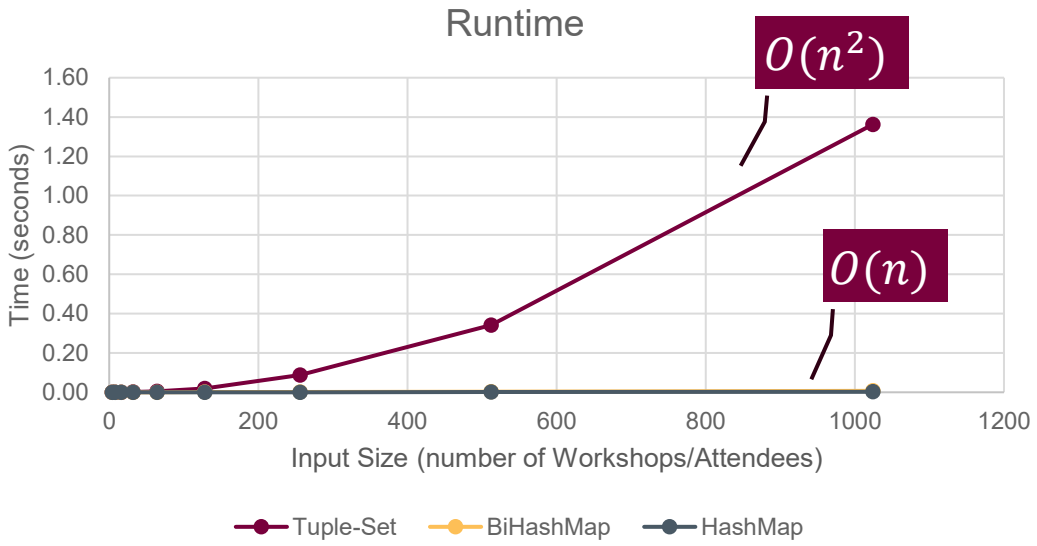
$$
\begin{array}{l}
numMeals \coloneqq 0 \\
\textbf{for}\ v, p' \in attends\ \textbf{do} \\
\quad \textbf{if}\ location(p') = room\ \textbf{then} \\
\quad\quad numMeals \coloneqq numMeals + 1
\end{array}
$$

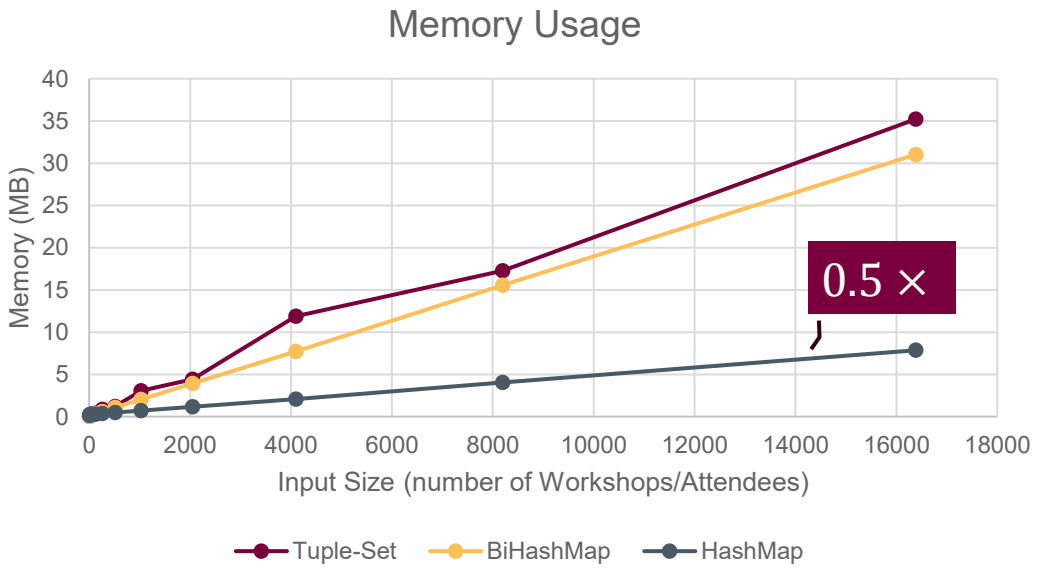
Post-condition:

- Code is imperatively executable

Preliminary Results

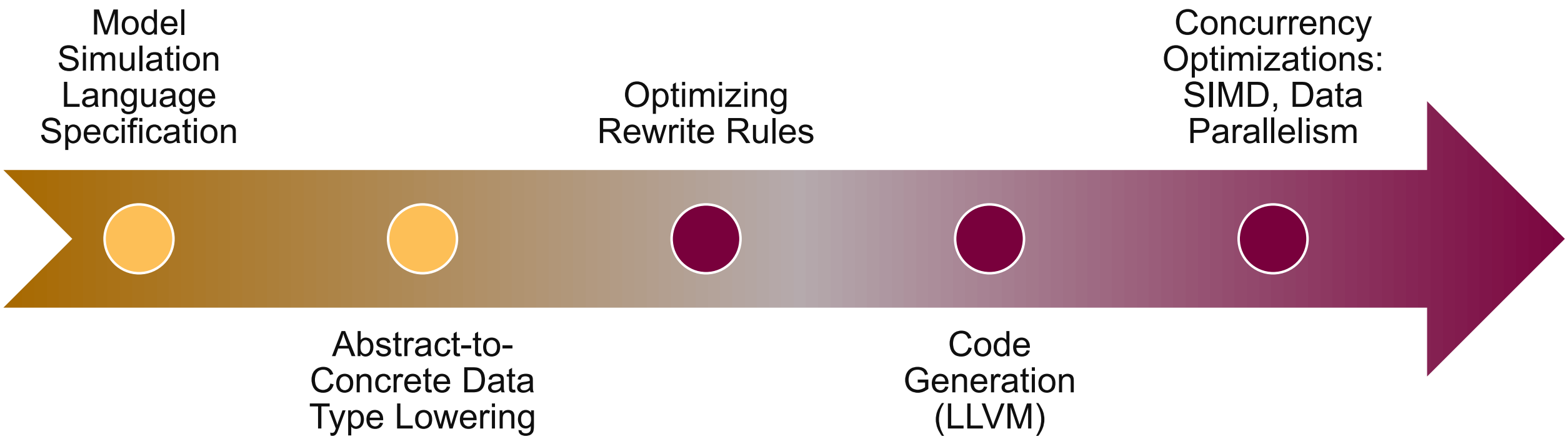
- Most operators only decrease the size of a resulting set
- Unions, Relation Overriding, and Cartesian Products are limited in growth
- Intermediate sets never grow larger than *max(starting sets, resulting set)*
- Optimizing Hashmaps:
    1. Expand operations into comprehensions
    2. Simplify large groups of ∧-predicates
    3. Eagerly apply conditions on sequential ∨-predicates





Current Compiler State and Roadmap

Development of a High-Level, Efficient, Set-Based Language



References

- Michael Leuschel. *ProB2 Jupyter Notebooks GitLab.* 2020. https://gitlab.cs.uni-duesseldorf.de/general/stups/prob2-jupyter-notebooks/-/blob/9d830112d9713d0cbc12d9d285f44ee61c827ccf/puzzles/Game_of_Life.ipynb
- Jean-Raymond Abrial. *Modeling in Event-B: system and software engineering*. Cambridge University Press, 2010.
- David Gries and Fred Schneider. *A Logical Approach to Discrete Math*. Springer New York, 1993.
- Robert Dewar. *The SetL Programming Language.* Bell Laboratories, 1979.
- Simon Peyton Jones, Andrew Tolmach, and Tony Hoare. *Playing by the rules: rewriting as a practical optimisation technique in GHC*. In: 2001 Haskell Workshop. ACM SIGPLAN. Sept. 2001.
- Michael Leuschel. *Programming in B: Sets and Logic all the Way Down.* In: LPOP 2022.
- Maximiliano Cristia and Gianfranco Rossi. *{log}: Programming and Automated Proof in Set Theory.* In: LPOP 2022.
- Douglas R. Smith and Stephen J. Westfold. *Transformations for Generating Type Refinements*. In: Formal Methods. FM 2019 International Workshops. Springer International Publishing, 2020, pp. 371-387
- Edmond Schonberg, Jacob T. Schwartz, and Micha Sharir. 1979. *Automatic data structure selection in SETL*. In Proceedings of the 6th ACM SIGACT-SIGPLAN symposium on Principles of programming languages (POPL '79). Association for Computing Machinery, New York, NY, USA, 197–210. https://doi.org/10.1145/567752.567771
- Stefan M. Freudenberger, Jacob T. Schwartz, and Micha Sharir. 1983. *Experience with the SETL Optimizer.* ACM Trans. Program. Lang. Syst. 5, 1 (Jan. 1983), 26–45. https://doi.org/10.1145/357195.357197

Thank you!

Extra

Example - Warehouse Inventory System

- Furniture material catalogue
- Warehouse inventory
- Product recipes

$$Material = \{2 \times 4\ Plank, Hex\ Bolt, 3\ Inch\ Screw, \ldots\}$$
$$Price = \mathbb{N}_0$$
$$Product = \{Cabinet, Desk, Bookshelf, \ldots\}$$

$$catalogue : Material \rightarrow Price$$
$$inventory : bag[Material]$$
$$recipes : Product \rightarrow bag[Material]$$

Equivalent to: $Material \nrightarrow \mathbb{N}$

Restocking price, with a target inventory $inventory_t$:

$$\sum p \mapsto n_m \cdot p \mapsto n_m \in catalogue^{-1} \circ (inventory_t - inventory) \mid p \times n_m$$

Bag Difference

Warehouse Inventory System – Translation

Many-to-One Total Function

$catalogue: Material \to Price$

Many-to-One Function

$inventory: bag[Material]$

$recipes: Product \to bag[Material]$

Restocking price, with a target inventory $inventory_t$:

$$price = \sum p \mapsto n_m \cdot p \mapsto n_m \in catalogue^{-1} \circ (inventory_t - inventory) \mid p \times n_m$$

$$price = \sum \boldsymbol{m \mapsto n_m} \cdot \exists n, m, p \cdot \boldsymbol{m \mapsto n_m \in inventory_t} \wedge n = n_m - inventory(m) \wedge n \geq 0 \wedge p = catalogue(m) \mid p \times n_m$$

$price := 0$
for $m, n_m \in inventory_t$ **do**
  $n := n_m - inventory(m)$
  if $n \geq 0$ **then**
    $price := price + catalogue(m) \times n$

Examples Running Time

	Core Expression	Naïve Running Time	Optimized Running Time
Visitor Information System	$(location^{-1} \circ attends^{-1})[\{room\}]$	$O(\|location\|\|attends\|)$	$O(\|attends\|)$
Warehouse Inventory System	$\sum p \mapsto n_m \cdot p \mapsto n_m \in (inventory_t - inventory) \circ catalogue^{-1} \mid p \times n_m$	$O(\|inventory_t\|\|catalogue\|)$	$O(\|inventory_t\|)$

Operation Running Time With (Bi-directional) HashMaps

Operation	Expected Running Time
$S \cup T$	$O(\|S\| + \|T\|)$
$S \cap T$	$O(min(\|S\|, \|T\|))$
$R[S]$	$O(min(\|S\|, \|R\|))$
$\mathrm{R} \circ Q$	$O(\|R\|\|Q\|)$
$(S \cup T) \cap U$	$O(min(\|S\| + \|T\|, \|U\|))$
$S \cap R[T]$	$O(min(\|S\|, \|R\|, \|T\|))$
$(\mathrm{R} \circ Q)[S]$	$O(min(\|S\|, \|R\|) + min(\|R[S]\|, \|Q\|))$

- Simplify large groups of ∧-predicates
- Eagerly apply conditions on sequential ∨-predicates